

# AGtest-framework

Analiza struktury projektu, kodu, aktualnej funkcjonalności, wyników weryfikacji oraz rekomendowanego kierunku dalszego rozwoju.

**Projekt:** pymoo\_gui\_framework

**Typ:** desktopowy framework eksperymentów MOEA

**Technologie:** Python, PyQt5, pymoo, Platypus, NumPy

**Data analizy:** 15 września 2026

**Zakres:** struktura, runtime, GUI, metryki, eksport, testy

## 1. Podsumowanie wykonawcze

Framework jest działającą aplikacją eksperymentalną do optymalizacji wielokryterialnej, a nie tylko prostym interfejsem do biblioteki `pymoo`. Ma własne rejestry komponentów, implementacje i adaptory algorytmów, benchmarki, warstwę metryk, eksport XLSX, wykonywanie wsadowe oraz wizualizację wyników.

|                         |                                     |                                  |
|-------------------------|-------------------------------------|----------------------------------|
| <b>17</b><br>algorytmów | <b>62</b><br>konfiguracje problemów | <b>8</b><br>metryk jakości       |
| <b>3</b><br>karty GUI   | <b>19</b><br>testów                 | <b>536</b><br>deklaracji runtime |

**Ocena:** zakres funkcjonalny jest już wystarczający do prowadzenia eksperymentów, lecz przed publikacją wyników najważniejsze jest wzmocnienie poprawności naukowej: zgodności algorytm–problem, konfiguracji HV, wielokrotnych uruchomień oraz kompletności metadanych.

## 2. Struktura projektu

| Element                                         | Odpowiedzialność                                                                                                     |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>src/pymoo_gui/app.py</code>               | Główne GUI, formularze, walidacja, sterowanie eksperymentem, tabele, komunikaty i wywołanie eksportu.                |
| <code>src/pymoo_gui/algorithms</code>           | Rejestr algorytmów, implementacje, adaptory oraz wspólny dispatcher <code>minimize</code> .                          |
| <code>src/pymoo_gui/algorithms/common.py</code> | Callback generacji, normalizacja populacji, wybór rozwiązań dopuszczalnych i niezdominowanych oraz wywołanie metryk. |
| <code>src/pymoo_gui/problems</code>             | Lokalne benchmarki, rejestr problemów, fabryki oraz ładowanie znanych frontów Pareto.                                |
| <code>src/pymoo_gui/metrics/quality.py</code>   | Obliczanie wskaźników jakości i obsługa brakujących lub niepoprawnych danych.                                        |
| <code>src/pymoo_gui/metrics/export.py</code>    | Generowanie arkuszy XLSX bez zależności od pandas lub openpyxl.                                                      |
| <code>src/pymoo_gui/multi.py</code>             | Wsadowe wykonywanie iloczynu problemów i algorytmów oraz zapis wyników.                                              |
| <code>src/pymoo_gui/parallel.py</code>          | Opcjonalna ewaluacja osobników przez pulę procesów lub wątków.                                                       |
| <code>src/pymoo_gui/viz</code>                  | Wizualizacja Pareto oraz przebiegów metryk.                                                                          |
| <code>tests</code>                              | Testy rejestrów, wybranych algorytmów, Multi, szablonu benchmarku i przebiegów metryk.                               |

Uruchomienie rozpoczyna się w `run_gui.py` lub przez entry point `AGtest-framework`. `MainWindow` buduje interfejs na podstawie rejestrów `PROBLEMS` i `ALGORITHMS`. Obliczenia odbywają się poza wątkiem GUI. Standardowe algorytmy są przekazywane do `pymoo.optimize.minimize`, natomiast adaptory i algorytmy badawcze mogą udostępnić własną metodę `gui_minimize`.

Historyczny pakiet `pymoo_gui.algoritms` pozostaje aliasem zgodności wstecznej dla poprawnego pakietu `pymoo_gui.algorithms`.

## 3. Aktualna funkcjonalność

### 3.1. Karta Main

Karta obsługuje kompletny pojedynczy eksperyment:

- wybór problemu i algorytmu z rejestrów;
- dynamiczne formularze parametrów generowane z sygnatur fabryk i metadanych `form_fields`;
- walidację liczb całkowitych, zmiennoprzecinkowych, wartości logicznych, wyborów i wartości ogólnych;
- konfigurację liczby generacji, ziarna losowego i trybu szczegółowych komunikatów;
- tryb `RAN`, w którym GUI nie narzuca limitu generacji;
- uruchamianie obliczeń w `QThread` i kooperacyjne zatrzymywanie;
- opcjonalną ewaluację równoległą przez procesy lub wątki;
- automatyczny lub ręczny punkt referencyjny hiperobjętości;
- podgląd znanego frontu Pareto, populacji oraz frontu niezdominowanego;
- ukrywanie frontu referencyjnego, pokazywanie populacji i automatyczne skalowanie osi;
- aktualizację wykresu w trakcie działania;
- tabelę metryk dla każdej zakończonej generacji;
- konsolę komunikatów i diagnostyki;
- automatyczny eksport metryk i rozwiązań.

### 3.2. Karta Multi

Karta wykonuje iloczyn kartezjański:

wybrane problemy × wybrane algorytmy = liczba uruchomień

Obsługuje wielokrotny wybór, wymagany górny limit generacji, wspólne ziarno, pasek postępu, status każdej kombinacji, anulowanie oraz kontynuowanie po błędzie pojedynczego uruchomienia. Wyniki już zakończonych lub częściowo wykonanych przebiegów są zachowywane. Każda kombinacja otrzymuje osobny plik metryk i rozwiązań. Karta celowo nie renderuje wykresów.

Aktualne ograniczenie: Multi używa domyślnych parametrów rejestru i nie pozwala ustawić innych parametrów dla poszczególnych algorytmów lub problemów.

### 3.3. Karta Metric trajectories

Karta działa wyłącznie dla bieżącego eksperymentu uruchomionego na stronie Main. Pokazuje osiem osobnych wykresów wartości względem generacji: IGD, GD, IGD+, GD+, Spread, Delta, HV i KKTPM. Brak poprawnej wartości jest oznaczany jako `N/A`. Eksperymenty Multi nie zmieniają tych wykresów.

### 3.4. Algorytmy

Aktualnie zarejestrowane są:

- AGE-MOEA, Eps-MOEA, Eps-NSGA-II, GDE3, HypE, IBEA;
- Learning-across-problems EDMO, LMOEA-DS, MOEAD;
- MOEA/D z rodziny epsilon przez Platypus;
- NSGA-II, NSGA-III, R-NSGA-III, RVEA, SPEA2;
- RNN-guided DMO oraz TS-NSGA.

Framework pozwala integrować standardowy obiekt `pymoo`, adapter biblioteki zewnętrznej albo własny algorytm z metodą `gui_minimize`. Dostępny jest również plik `empty_algorithm_template.py` jako punkt startowy nowej implementacji.

### 3.5. Problemy benchmarkowe

| Rodzina | Liczba konfiguracji | Zakres                                                                                                   |
|---------|---------------------|----------------------------------------------------------------------------------------------------------|
| Lokalne | 12                  | Binh2, ConstrEx, FON, Golinski, Kursawe, Osyczka2, Schaffer, Srinivas, Tanaka, Viennet2, Viennet3, Water |
| ZDT     | 6                   | ZDT1–ZDT6                                                                                                |
| DTLZ    | 7                   | DTLZ1–DTLZ7                                                                                              |
| LZ09    | 9                   | LZ09_F1–LZ09_F9                                                                                          |
| UF      | 10                  | UF1–UF10                                                                                                 |
| WFG     | 18                  | WFG1–WFG9, każdy osobno w 2D i 3D                                                                        |

Łącznie daje to 62 konfiguracje i 53 unikalne benchmarki. Plik `empty_benchmark_template.py` zawiera klasę zgodną z `pymoo`, walidację, wektorową funkcję `_evaluate`, miejsca rozszerzeń dla ograniczeń, fabrykę, generator frontu oraz komplet metadanych formularza.

### 3.6. Metryki i warunki dostępności

| Metryka    | Warunki i obecne zachowanie                                                                                         |
|------------|---------------------------------------------------------------------------------------------------------------------|
| HV         | Wymaga punktu referencyjnego o wymiarze zgodnym z liczbą celów.                                                     |
| GD / IGD   | Wymagają znanego frontu Pareto i zgodnej liczby celów.                                                              |
| GD+ / IGD+ | Jak wyżej; wykorzystują implementacje wskaźników z <code>pymoo</code> .                                             |
| Spread     | Obliczany z bieżącego zbioru celów; implementacja jest znormalizowaną średnią odległości par punktów.               |
| Delta      | Obecnie uruchamiana tylko dla klucza algorytmu <code>nsga2</code> , problemu 2D i dostępnego frontu referencyjnego. |
| KKTPM      | Wymaga zmiennych decyzyjnych i problemu; wykorzystuje pochodne analityczne albo różnice skończone.                  |

Callback filtruje rozwiązania niedopuszczalne, wyznacza front niezdominowany i dopiero na nim oblicza wskaźniki. Do GUI trafiają również `n_gen`, `n_eval` i `n_nds`.

### 3.7. Eksport

Framework buduje minimalne skoroszyty XLSX bez pandas i openpyxl. Plik metryk zawiera historię generacji, ewaluacji, liczby rozwiązań niezdominowanych i wskaźników. Plik rozwiązań zawiera identyfikator, wartości `f1...fN` oraz — gdy są dostępne — zmienne `x1...xN`.

Dla pojedynczego eksperymentu można zapisywać rozwiązania dla ostatniej generacji, co N generacji albo według schematu kaskadowego. Nazwy plików są zabezpieczane przed prostym nadpisaniem przez dodawanie kolejnych sufiksów.

## 4. Wyniki weryfikacji

- Wszystkie **19 testów** repozytorium przechodzą.
- Wszystkie **62 fabryki problemów** utworzyły problem i wykonały podstawową ewaluację bez błędu.
- **58 problemów** udostępnia poprawny znany front Pareto.
- Znany front nie jest obecnie dostępny dla ZDT5, DTLZ5, DTLZ6 i DTLZ7.
- Wszystkie **17 algorytmów** wykonało dwie generacje na problemie Schaffera.

Dodatkowy test 68 reprezentatywnych połączeń obejmował ZDT5, DTLZ2, WFG1 3D i Water. Uzyskano 58 niepustych wyników. Cztery krótkie przebiegi Water zakończyły się bez znalezienia rozwiązania dopuszczalnego, a sześć konfiguracji ujawniło rzeczywistą niezgodność lub niebezpieczną konfigurację domyślną.

Smoke test potwierdza integrację techniczną, ale nie stanowi dowodu, że implementacja każdego algorytmu jest matematycznie identyczna z publikacją źródłową. Do takiego potwierdzenia potrzebne są testy referencyjne i eksperymenty statystyczne.

## 5. Problemy i rekomendacje

### Priorytet 1 Zgodność algorytm–problem

Karta Multi pozwala zaznaczyć dowolną kombinację, lecz nie każda kombinacja jest obsługiwana:

- R-NSGA-III ma domyślne punkty referencyjne 2D i nie uruchamia się dla 3 lub 5 celów.
- Implementacja `pymoo.MOEA/D` nie obsługuje problemów z ograniczeniami, takich jak Water.
- NSGA-III dla pięciu celów i `n_partitions=12` generuje 1820 kierunków przy populacji 100.
- RVEA z `n_partitions=99` dla pięciu celów może wygenerować ponad 4,4 mln kierunków i spowodować błąd pamięci.

Należy dodać deklaratywną macierz możliwości, walidację przed startem, limit liczby kierunków oraz automatyczne dopasowanie parametrów do `n_obj` i rozmiaru populacji. GUI powinno blokować lub wyraźnie oznaczać niewspierane pary przed eksperymentem.

Priorytet 1

## Poprawny punkt referencyjny HV

Automatyczny punkt Main to obecnie wektor wartości 1.1. Multi korzysta z podobnej funkcji z wyjątkiem specjalnego przypadku DTLZ1. Taki punkt nie jest uniwersalny dla WFG, Kursawe, Water i części innych benchmarków.

Punkt HV powinien być zdefiniowany w metadanych problemu, wspólny dla wszystkich porównywanych algorytmów, zapisany w wynikach i zweryfikowany względem skali celów. Nie powinien być dobierany osobno na podstawie końcowego wyniku każdego algorytmu, ponieważ zniszczyłoby to porównywalność.

Priorytet 1

## Powtórzenia i statystyka

Jeden seed jest niewystarczający do naukowego porównania algorytmów stochastycznych. Multi powinno przyjmować listę seedów albo liczbę powtórzeń, a raport zbiorczy powinien zawierać średnią, medianę, odchylenie standardowe, kwartyle, przedziały ufności, ranking i odpowiednie testy statystyczne.

Priorytet 1

## Formalizacja metryk

Dla każdej metryki należy udokumentować wzór, źródło bibliograficzne, kierunek optymalizacji, wymagane dane oraz zachowanie przy pustym froncie. Szczególnie ważne jest rozróżnienie obecnej definicji Spread od klasycznych definicji z literatury oraz jawne opisanie ograniczenia Delta.

Każda metryka powinna mieć test z ręcznie obliczonym wynikiem, test skalowania, test wartości nieskończoności i test braku wymaganego frontu.

Priorytet 2

## Testy automatyczne

Obecne testy obejmują rejestry, trzy algorytmy uruchamiane na Schafferze, podstawy Multi, szablon benchmarku i przebiegi metryk. Brakuje bezpośrednich testów wszystkich wskaźników, większości algorytmów, wszystkich benchmarków, ewaluacji równoległej, trybu RAN, anulowania, wizualizacji Pareto i kompletnej macierzy zgodności.

Potrzebne są również testy budowania i instalacji paczki, zwłaszcza sprawdzenie, czy wszystkie pliki `.pf` trafiają do wheel oraz sdist.

Priorytet 2

## Reprodukowalność wyników

Każdy eksport powinien zawierać arkusz `Metadata` lub osobny manifest z następującymi informacjami:

- algorytm, problem i wszystkie ich parametry;
- seed, limit generacji i punkt HV;
- czas rozpoczęcia, czas zakończenia i czas wykonania;
- status, komunikat błędu i liczba wykonanych ewaluacji;
- wersje Pythona, frameworka, pymoo, NumPy i Platypus;
- opcjonalny identyfikator commita Git.

Obecnie konfiguracja jest widoczna głównie w konsoli i nie jest w pełni trwale związana z wynikiem.

Priorytet 2

## Podział app.py

`app.py` ma około 2190 wierszy i łączy budowę GUI, modele formularzy, walidację, kontrolę wątków, prezentację, eksport oraz translację tekstów. Naturalny podział obejmuje:

- `ui/main_tab.py`, `ui/multi_tab.py`, `ui/trajectories_tab.py`;
- `forms.py` i `validation.py`;

- `run_controller.py` i `experiment_config.py`;
- `result_model.py` i `export_service.py`.

Logika filtrowania argumentów, budowania tabel rozwiązań i normalizacji komórek XLSX jest powielona pomiędzy Main i Multi. Powinna mieć jedną wspólną implementację.

### Priorytet 3 Nazewnictwo i teksty

Oś przebiegów używa już poprawnego terminu `Generation`, ale w pozostałej części GUI i eksporcie nadal występują `Maximum epochs`, `Last epoch`, `Every N epochs` i nazwy arkuszy `Epoch N`. Dla spójności MOEA warto wszędzie zastosować termin `Generation`.

Warstwa `translate_ui_text` kompensuje historyczne polskie lub uszkodzone teksty. Docelowo źródła powinny zawierać od razu poprawne angielskie komunikaty UTF-8, bez naprawiania ich w czasie działania.

### Priorytet 3 Wydajność i odporność zapisu

- KKTPM z różnicami skończonymi może wymagać wielu dodatkowych ewaluacji w każdej generacji.
- Częsty eksport frontów wielokrotnie przepisuje cały skoroszyt.
- XLSX jest zapisywany bezpośrednio do ścieżki docelowej; bezpieczniejszy byłby plik tymczasowy i atomowa zamiana.
- Równoległa ewaluacja wiersz po wierszu może być wolniejsza dla tanich, wektorowych funkcji NumPy.
- Zatrzymanie następuje przy callbacku, więc bardzo długa generacja nie zostanie przerwana natychmiast.
- Błędy wskaźników są często zamieniane na brak wartości; diagnostyka powinna podawać dokładną przyczynę `N/A`.

## 6. Rekomendowane przyszłe funkcje

---

- wykresy metryk względem **Function Evaluations**;
- porównanie wielu algorytmów na jednym wykresie;
- skala logarytmiczna i wybór widocznych metryk;
- eksport wykresów do PNG, SVG i PDF;
- indywidualne parametry algorytmów i problemów w Multi;
- lista seedów, liczba powtórzeń i agregacja statystyczna;
- wznawianie przerwanych eksperymentów;
- zapis i odczyt konfiguracji eksperymentu;
- wybór katalogu wynikowego;
- zbiorczy raport CSV/XLSX/PDF;
- kontrolowane równoległe wykonywanie niezależnych uruchomień Multi;
- parallel coordinates, scatter-matrix lub inne widoki dla więcej niż trzech celów;
- wyszukiwanie i filtrowanie list problemów oraz algorytmów;
- raport kompatybilności przed uruchomieniem eksperymentu.

## 7. Zalecana kolejność rozwoju

---

1. **Macierz zgodności i bezpieczne kierunki referencyjne.**

2. **Poprawne, problemowe punkty HV.**
3. **Wiele seedów, agregacja oraz metadane eksperymentów.**
4. **Testy referencyjne metryk i całej macierzy komponentów.**
5. **Podział app.py i usunięcie powielonej logiki.**
6. **Ujednolicenie terminologii i tekstów.**
7. **Nowe funkcje wizualne, FE i raporty porównawcze.**

Najważniejsza sekwencja prac: **zgodność kombinacji** → **poprawny HV** → **powtórzenia i metadane** → **testy metryk** → **refaktoryzacja GUI** → **nowe wizualizacje**.

## 8. Materiały projektu

---

- `README.md` — opis instalacji, architektury i rozszerzania frameworka.
- `docs/FUNCTION_CATALOG.md` — pełny katalog funkcji, metod, funkcji zagnieżdżonych i lambda.
- `src/pymoo_gui/algorithms/empty_algorithm_template.py` — szablon algorytmu.
- `src/pymoo_gui/problems/empty_benchmark_template.py` — szablon benchmarku.

Raport powstał na podstawie analizy lokalnego kodu źródłowego, konfiguracji projektu, rejestrów, testów i krótkich testów diagnostycznych. Nie zmienia on kodu wykonawczego frameworka.